



BRAD DEVICES

Built in Africa, for the World

BradGfx Architecture Specification

Torox G1 Microarchitecture — BradVector, BradRT & Neuro-Stream

Revision 1.0 — July 2026

Confidential — Brad Devices Proprietary

Contents

1	Introduction	3
1.1	Reference SIMT Runtime (Shipped)	3
2	BradVector Core Microarchitecture	3
2.1	Overview	3
2.2	Pipeline Stages	4
2.3	Warp Scheduler Details	5
2.4	Memory Hierarchy	5
3	BradRT Core Microarchitecture	6
3.1	Ray Tracing Pipeline	6
3.2	BVH Traversal Unit	6
3.3	Intersection Processing Unit	6
3.4	NeRF Accelerator	6
4	Neuro-Stream Core Microarchitecture	8
4.1	AI Upscaling Pipeline	8
4.2	Quality Modes	8
5	Memory System	9
5.1	Address Space	9
5.2	Infinity Buffer	9
6	Command Submission Model	9
7	Interrupt Model	9
8	SKU Configurations	10
9	Register Map Reference	11
10	Revision History	11

1 Introduction

Torox G1 is the microarchitecture that powers the entire Brad GPU family — BradGfx (discrete), BradFx (integrated), and BradApex (datacenter). It is the architectural brand for the generation, in the same way NVIDIA names a family “Pascal” or “Ada”; it is *not* a single core. Within each Torox G1 GPU, three specialised processor arrays do the work:

- **BradVector Cores**: General-purpose dual-issue superscalar wide-vector compute engines handling rasterization, shading, physics, and general compute (the CUDA-core-like workhorses).
- **BradRT Cores**: Dedicated ray tracing accelerators with BVH traversal, ray-primitive intersection, and NeRF evaluation.
- **Neuro-Stream Cores** (BradGfx) / **Neuro-Lite Cores** (BradFx): On-GPU AI accelerators for BradSense-Render.

1.1 Reference SIMT Runtime (Shipped)

The BradVector warp model is not paper. A reference implementation of the vector ISA ships in the Brad Devices repository at SIMT-VM level — 32-lane warps with per-lane predication and a uniform program counter, exactly the model §2 defines:

- **bradc**: reference assembler (`‘.bvbs’` → `‘.bvbc’`)
- **BVRT**: the runtime/simulator that executes assembled kernels
- **bradgdb**: breakpoints + S/V/P register and PC inspection
- **bradtimeline**: per-cycle trace with CSV export
- **braddev**: the unified one-binary CLI wrapping all of the above (`cvt`, `run`, `dbg`, `gpu`, `timeline`, `test`)

Verified kernels include SAXPY (float ALU over integer-address data, zero traps) and iterative Fibonacci ($\text{fib}(8) = 21$); the CLI self-test also links the SPMP/Fabric/EROE driver set, proving the toolchain boots the reference driver stack.

2 BradVector Core Microarchitecture

2.1 Overview

Each BradVector Core is a dual-issue, superscalar, wide-vector processor. It executes a 32-thread warp in SIMT fashion, with each thread having its own 32-entry register file.

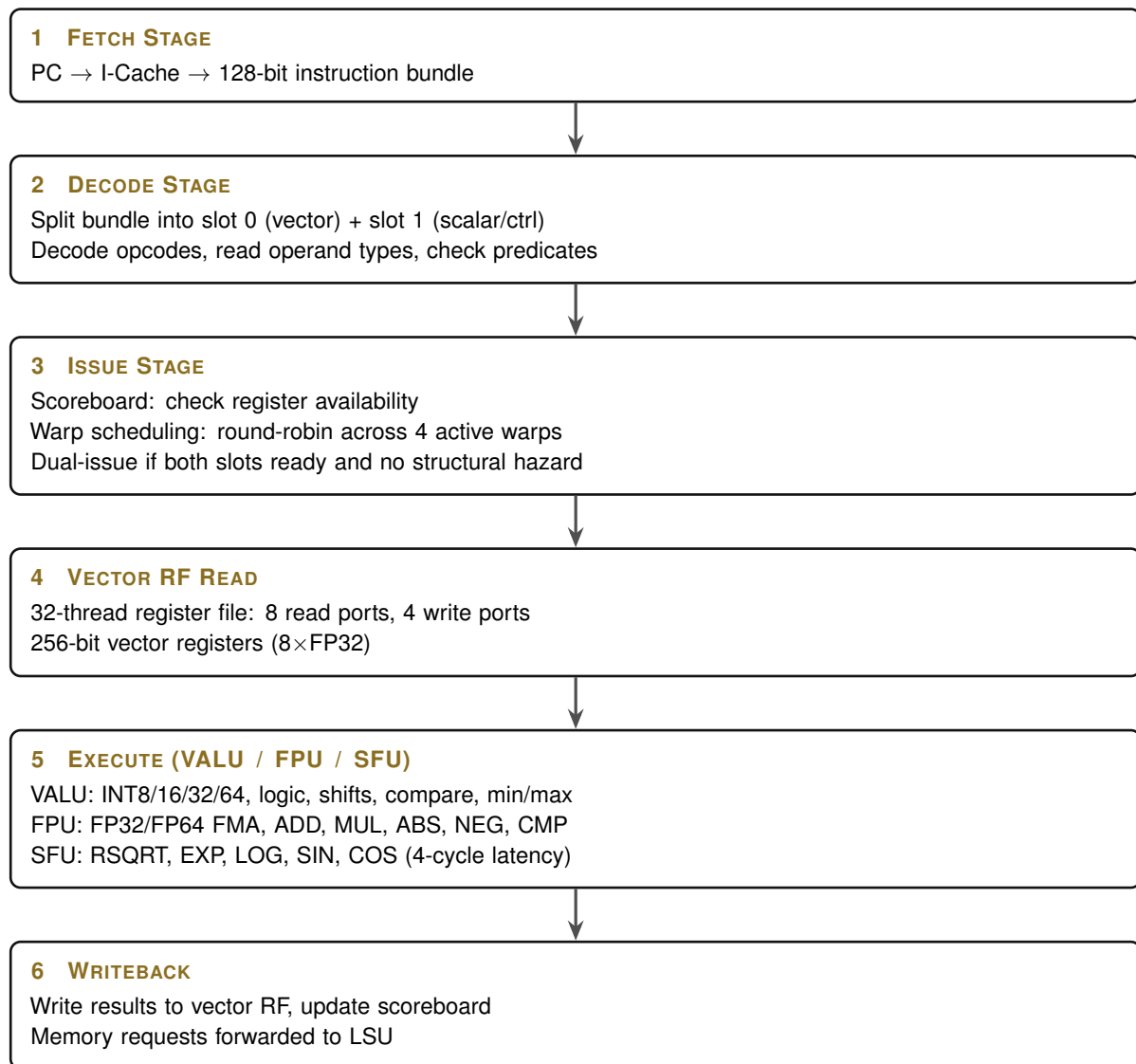


Figure 1: BradVector Core pipeline stages

2.2 Pipeline Stages

Fetch: The instruction cache (8 KB, 4-way set-associative, 64-byte line) supplies 128-bit instruction bundles each cycle. The fetch unit maintains a separate PC per warp (4 warps × 32-bit PC). Branch prediction uses a 2-bit saturated counter per 256-entry BTB.

Decode: Each 128-bit bundle contains two 64-bit instruction slots. Slot 0 is always a vector operation (VALU/FPU/memory). Slot 1 is a scalar ALU operation, a control-flow op (branch/predicate), or a second VALU if the bundle is vector-paired. Decode checks:

- Opcode validity and operand encoding
- Warp predicate (per-thread active mask from SIMT stack)
- Register scoreboard for RAW hazards

Issue: The warp scheduler selects from 4 active warps using round-robin arbitration. In each cycle it attempts to issue both slots of the selected warp. If only one slot is ready (scoreboard stall on the other), it issues a single slot and compacts the bundle. Issue logic checks:

- Scoreboard: destination registers must not be busy
- Structural: VALU and FPU are separate units — can both issue
- Memory: at most one memory operation per cycle

Vector Register File: $32 \text{ threads} \times 32 \text{ registers} \times 256 \text{ bits} = 32 \text{ KB}$ per warp. 4 warps = 128 KB total register file. 8 read ports (4 per slot $\times$ 2 slots) and 4 write ports. Each register holds $8 \times \text{FP32}$, $4 \times \text{FP64}$, or $16 \times \text{INT16}$.

Execute:

- **VALU:** 8-lane SIMD integer ALU. Supports ADD, SUB, MUL, MIN, MAX, CMP, ABS, NEG, AND, OR, XOR, shift, bit reverse. All operations 1-cycle latency, fully pipelined.
- **FPU:** 8-lane SIMD floating-point. IEEE 754-2019 compliant. FMA: 4-cycle latency. ADD/MUL: 3-cycle. CMP/MIN/MAX: 2-cycle.
- **SFU:** Special function unit. RSQRT, EXP, LOG, SIN, COS. 4-cycle latency, not fully pipelined (4-entry FIFO).
- **LSU:** Load/store unit. Handles gather/scatter, atomic operations, barrier synchronization. Connects to L1D cache.

Writeback: Results are written to the vector register file. The scoreboard is updated to mark destinations ready. Memory loads update the scoreboard when the LSU completes (variable latency).

2.3 Warp Scheduler Details

- 4 warps per core, 32 threads per warp = 128 threads/core
- Scheduler state per warp: PC, active mask (predicate stack), scoreboard bitmap (32 bits for 32 registers)
- Round-robin selection with priority boost for memory-dependent warps (warp that was waiting on load gets selected next when data arrives)
- On branch divergence: push new active mask to predicate stack, execute taken path, pop, execute not-taken path

2.4 Memory Hierarchy

Table 1: BradVector memory hierarchy per core

Level	Size	Associativity	Latency
L1 Instruction Cache	8 KB	4-way	2 cycles
L1 Data Cache	16 KB	4-way	3 cycles (hit)
Local Memory (scratchpad)	64 KB	—	1 cycle (banked)

L1 misses go to the shared L2 cache (256 KB, 8-way, 12-cycle latency) which connects to the BradFusion Fabric for SPMP access.

3 BradRT Core Microarchitecture

3.1 Ray Tracing Pipeline

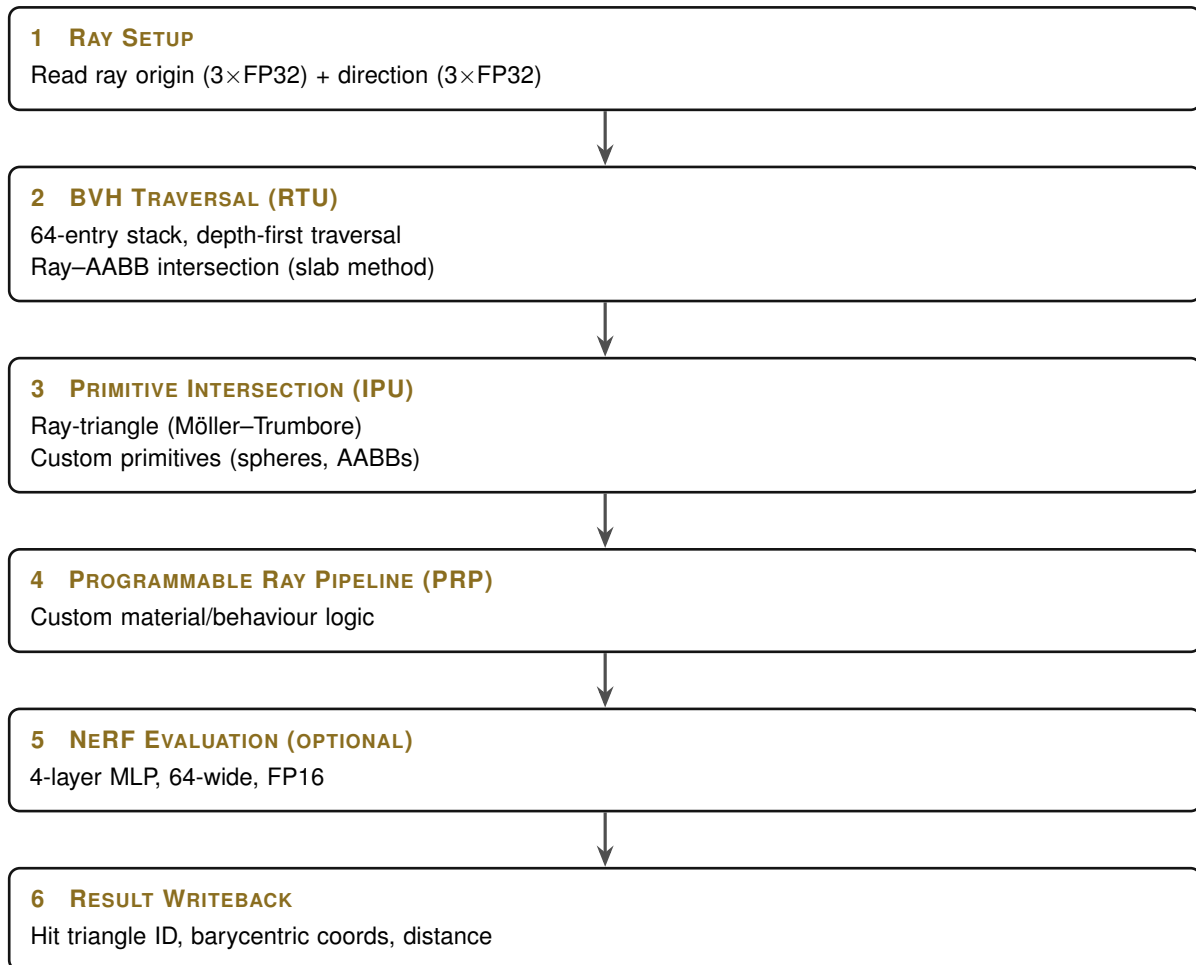


Figure 2: BradRT Core ray tracing pipeline

3.2 BVH Traversal Unit

- 64-entry hardware stack for depth-first BVH traversal
- Each node: 6×FP32 AABB (min/max per axis) + child pointers
- 2-cycle per node for AABB intersection test
- 4 KB BVH cache (holds 64 nodes)
- Early termination on closest-hit distance

3.3 Intersection Processing Unit

- 4 parallel ray-triangle testers (Möller–Trumbore)
- 8-cycle latency per triangle test, fully pipelined
- Supports custom primitives via PRP microcode
- Configurable intersection mask (triangles, spheres, AABBs only)

3.4 NeRF Accelerator

A small fixed-function MLP for Neural Radiance Field evaluation:

- 4 layers: 64 → 64 → 64 → 4 (RGBA + density)
- FP16 arithmetic with INT8 weight storage
- 64 MAC units, fully pipelined

- 20-cycle latency per evaluation point

4 Neuro-Stream Core Microarchitecture

4.1 AI Upscaling Pipeline

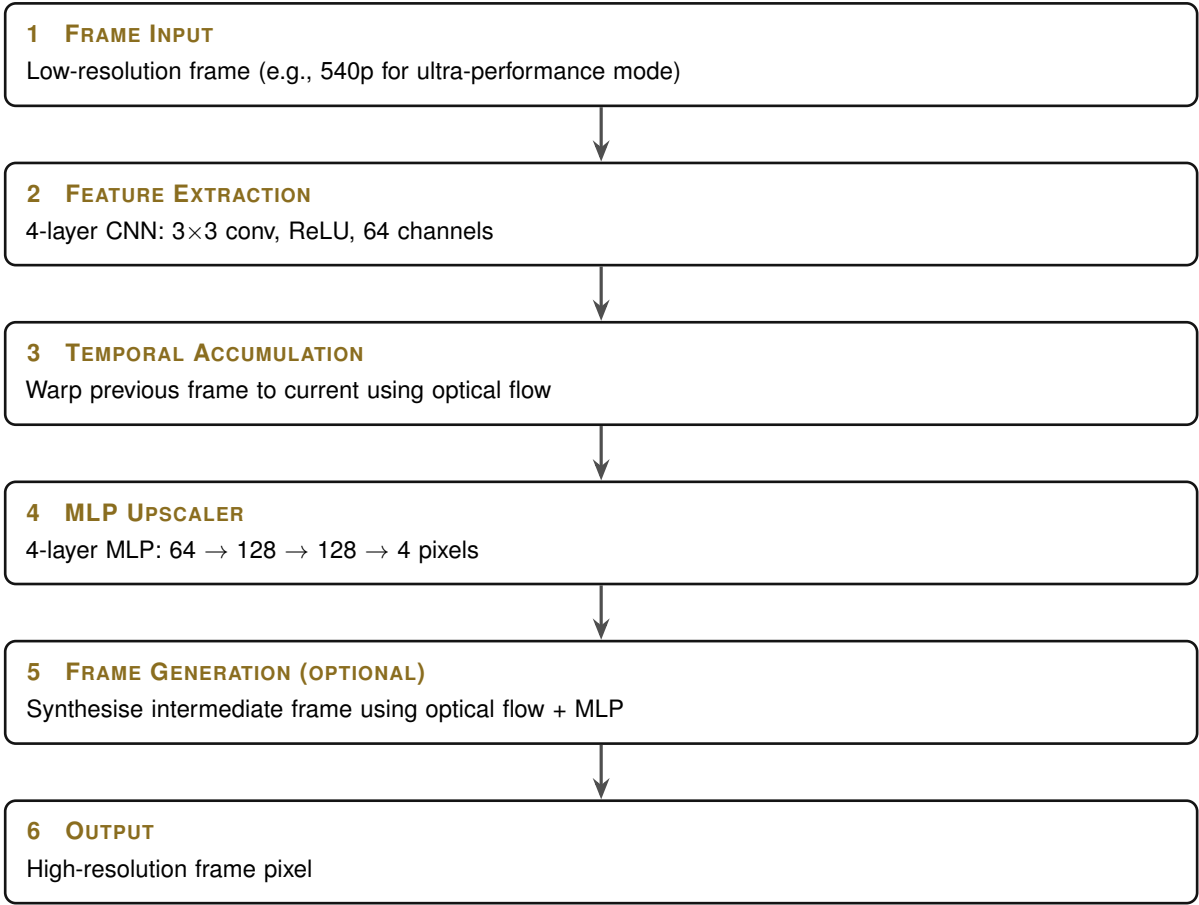


Figure 3: Neuro-Stream upscaling pipeline

4.2 Quality Modes

Table 2: Neuro-Stream quality/performance tradeoffs			
Mode	Render Resolution	Target	NSU Load
Quality	1440p	4K	60%
Balanced	1080p	4K	40%
Performance	720p	4K	25%
Ultra Performance	540p	4K	15%
Native AA	4K	4K	10%

5 Memory System

5.1 Address Space

Table 3: BradGfx address space

Region	Size	Description
GPU-local L2	256 KB–2 MB	Shared L2 cache
VRAM	8–48 GB	GDDR7 8–32 GB (Max Xtensive: 32 GB on 192–512-bit bus); HBM3e 48 GB (Max ANF)
Infinity Buffer	0–2 GB	On-die SRAM LLC (BradGfx 700+)
SPMP fabric	Up to 128 GB	Unified memory pool (zero-copy)
MMIO	4 MB	Control registers

5.2 Infinity Buffer

BradGfx 700+ includes a 3D-stacked SRAM tile acting as a massive on-package cache. It absorbs frame-buffer traffic and reduces SPMP accesses by $\sim 40\%$. Configurable as:

- LRU cache (default): evicts oldest frame tiles
- Pinned region: developer-locked memory for critical assets
- Direct-allocation: scratchpad mode for compute kernels

6 Command Submission Model

Applications submit work via a ring buffer in SPMP memory:

1. Driver writes a command packet to the ring buffer at tail pointer
2. Driver advances tail pointer (write to `CMD_QUEUE_TAIL`)
3. Driver rings doorbell (write to `CMD_QUEUE_DOORBELL`)
4. GFX frontend reads packet from ring buffer
5. Hardware executes the command (draw, dispatch, copy, barrier)
6. Completion interrupt fires; driver processes in ISR

7 Interrupt Model

Table 4: BradGfx interrupt sources

IRQ	Source	Description
0	GFX_CMD_DONE	Command buffer complete
1	GFX_PAGE_FAULT	SPMP page fault
2	GFX_ILLEGAL_INSTR	Illegal instruction
3	GFX_TIMEOUT	Shader timeout
4	NSU_FRAME_DONE	Frame reconstruction complete
5	NSU_FG_DONE	Generated frame complete
6	RT_TRACE_DONE	Ray trace complete
7	SEC_HDCP_FAIL	HDCP authentication failure
8	THERMAL_THROTTLE	Thermal throttling active

8 SKU Configurations

Table 5: BradGfx SKU hardware parameters

SKU	BradVector	BradRT	Neuro-Stream	L2	VRAM	TDP
Gfx 100	64	16	16	256 KB	8 GB	75W
Gfx 300	128	32	32	512 KB	12 GB	130W
Gfx 500	256	64	64	768 KB	16 GB	220W
Gfx 700	512	128	128	1 MB	24 GB	350W
Gfx Max	1024	256	192	2 MB + 2 GB IB	48 GB	600W
Max Xtensive	256	128	512	2 MB + 2 GB IB	32 GB GDDR7	225W

9 Register Map Reference

Table 6: Key BradGfx MMIO registers (base 0x4000_0000)

Offset	Name	Width	Description
0x0000	GFX_CTRL	32	Global enable, VRS, NSU, RT enables
0x0004	GFX_STATUS	32	Busy/ready flags
0x0008	GFX_REVISION	32	IP revision
0x000C	GFX_SKU_ID	32	0=Lite, 1=Std, 2=Ultra, 3=Max, 4=Xntensive
0x0010	GFX_NUM_VEC	32	Number of BradVector cores
0x0014	GFX_NUM_RT	32	Number of BradRT cores
0x0018	GFX_NUM_NSU	32	Number of Neuro-Stream units
0x001C	GFX_FABRIC_BW	32	Fabric bandwidth MB/s
0x0020	GFX_CLK_GATE	32	Per-engine clock gate mask
0x0024	GFX_PWR_STATE	32	Power state P0–P4
0x0028	GFX_PWR_LIMIT	32	Power limit in watts
0x003C	GFX_INTR_ENABLE	32	Interrupt enable mask
0x0040	GFX_INTR_STATUS	32	Interrupt status (W1C)
0x0044	GFX_PERF_CNTR_0	64	Performance counter 0
0x004C	GFX_PERF_CNTR_1	64	Performance counter 1
0x1000	CMD_QUEUE_DOORBELL	32	Ring doorbell
0x1004	CMD_QUEUE_HEAD	32	HW head pointer
0x1008	CMD_QUEUE_TAIL	32	SW tail pointer
0x100C	CMD_QUEUE_BASE	64	Queue base in SPMP
0x1014	CMD_QUEUE_SIZE	32	Queue entries

10 Revision History

Rev	Date	Changes
1.0	July 2026	Initial BradVector/BradRT/Neuro-Stream arch spec
1.1	Sept 2026	Software receipts: reference SIMT runtime shipped — bradc assembler, BVRT simulator, bradgdb, bradtimeline, unified braddev CLI; SAXPY & fib verified zero-trap